



COMP 2067

Introduction to

Formal Reasoning

Lecture 11 Trees

Author: Prof. Thorsten Altenkirch

Adapted by: Dr. Kweh Yeah Lun



University of
Nottingham

UK | CHINA | MALAYSIA

Learning Outcomes

At the end of this lecture, you will be able to

- understand the inductive definition of trees
- understand the working principle of the tree sort



Introduction to Trees

- Trees are everywhere, there isn't just one type of trees but many datatypes that can be subsumed under the general concept of trees.
- Indeed, the types discussed in the previous chapters (natural numbers and lists) are special instances of trees.
- The nodes of the tree are labelled with numbers.



- Example: Define the type of binary trees with nodes labelled with natural numbers

Try it!

```
inductive Tree : Type
| leaf : Tree
| node : Tree → N → Tree → Tree
```



- The definition has **two constructors, leaf and node**, which are used to construct **elements of the Tree type**.
- **leaf**: represents a **leaf node** in a tree structure.
- **node**: represents a **non-leaf node** in a tree structure.
- It takes **three arguments**:
 - The **first argument is another tree**, which represents the **left subtree**.
 - The **second argument is a natural number ($\mathbb{N}$)**, typically representing **some value associated with the node**.
 - The **third argument is another tree**, representing the **right subtree**.



- By **using these constructors recursively**, complex tree structures can be built where **each node can have two subtrees (left and right) or be a leaf with no subtrees.**
- **Each node has at most two children.**



University of
Nottingham

UK | CHINA | MALAYSIA

Tree Sort

- leverages the properties of a Binary Search Tree (BST) to sort a collection of comparable items.
- Two main steps: Building the Binary Search Tree, and In-order Traversal of the BST



1. Building the Binary Search Tree

- Iterate through the input collection (e.g., an array or a list).
- For each item, insert it into an initially empty Binary Search Tree.
- **If the tree is empty, the new item becomes the root.**
- **If the new item is less than the current node's value, it is inserted into the left subtree.**
- **If the new item is greater than or equal to the current node's value, it is inserted into the right subtree.**
- This process is repeated recursively until the correct position for the new item (as a new leaf node) is found.



2. In-order Traversal of the BST

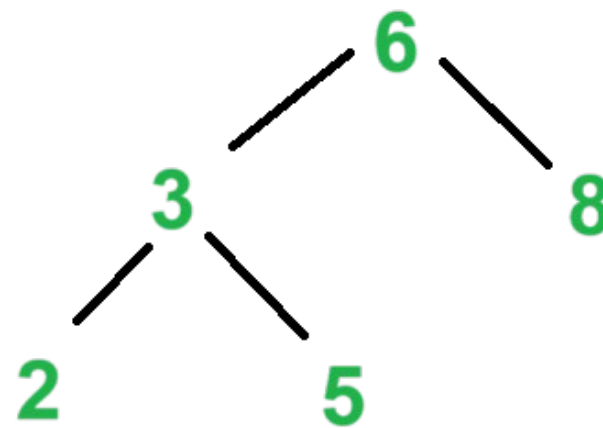
- Once all the items from the input collection have been inserted into the BST, perform an in-order traversal of the tree.
 - **Traverse the left subtree.**
 - **Visit the root node.**
 - **Traverse the right subtree.**
- The in-order traversal will **visit the nodes in ascending sorted order.**
- As each node is visited during the in-order traversal, its value is collected to form the sorted output.



Example: sort the list [6, 3, 8, 2, 5]

Building the BST:

- Insert 6: (6) (root)
- Insert 3: (3) <- 6
- Insert 8: 6 -> (8)
- Insert 2: (2) <- 3 <- 6 -> 8
- Insert 5: 2 <- 3 -> (5) <- 6 -> 8





In-order Traversal:

- Traverse left subtree of 6:
 - Traverse left subtree of 3:
 - Traverse left subtree of 2:
(empty)
 - Visit 2. Output: [2]
 - Traverse right subtree of 2:
(empty)
 - Visit 3. Output: [2, 3]
 - Traverse right subtree of 3:
 - Traverse left subtree of 5:
(empty)
 - Visit 5. Output: [2, 3, 5]
 - Traverse right subtree of 5:
(empty)
- Visit 6. Output: [2, 3, 5, 6]
- Traverse right subtree of 6:
 - Traverse left subtree of 8:
(empty)
 - Visit 8. Output: [2, 3, 5, 6, 8]
 - Traverse right subtree of 8:
(empty)
- The final sorted output is **[2, 3, 5, 6, 8]**.



University of
Nottingham

UK | CHINA | MALAYSIA

Next lecture...

- **No more next lecture!!!** ◀◀



- Give yourself a big clap.
- Survived once again? Great! ◀◀
- Any question?



“Code is read much more often than it is
written.”

– Guido van Rossum

Hope you all enjoy this lecture!